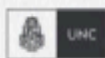


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Harbor



WE WANT
SKILLS!

mundosE

Manual para el alumno: Harbor.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Introducción: el rol de Harbor en la cadena DevOps.....	3
2. Arquitectura lógica en Kubernetes y conceptos de exposición.....	4
3. Gobierno y seguridad de la cadena de suministro.....	6
4. Versionado, reproducibilidad y promoción inmutable.....	7
5. Integración conceptual con CI/CD.....	8
6. Operación, gobernanza y riesgo.....	10
Bibliografía.....	12

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Harbor

Objetivo del encuentro:

Brindar al estudiantado una comprensión teórica, clara y aplicada de Harbor como **registro de artefactos OCI**, su valor en Kubernetes y en la cadena **CI/CD**, y los fundamentos de **gobierno, seguridad** y **trazabilidad** que habilita. Este manual evita pasos operativos y comandos, focalizando en criterios conceptuales y en la lectura crítica de evidencias (digest, firmas, SBOM y reportes de escaneo) para la toma de decisiones.

Contenido a desarrollar:

- ✓ Introducción: el rol de Harbor en la cadena DevOps
- ✓ Arquitectura lógica en Kubernetes y conceptos de exposición
- ✓ Gobierno y seguridad de la cadena de suministro
- ✓ Versionado, reproducibilidad y promoción inmutable
- ✓ Integración conceptual con CI/CD
- ✓ Operación, gobernanza y riesgo

1. Introducción: el rol de Harbor en la cadena DevOps

Harbor es un **registro de artefactos** alineado con las especificaciones de la **Open Container Initiative (OCI)** que cumple un papel central en arquitecturas cloud-native: **almacena, organiza, protege y distribuye** imágenes de contenedores y otros artefactos compatibles con OCI. En términos teóricos, un registro no es un simple repositorio de archivos; es una **fuentes de verdad** con controles de acceso, políticas y auditoría, de la cual dependen los entornos de ejecución (Kubernetes, orquestadores, plataformas de serverless, así como procesos automatizados de CI/CD).

El valor de Harbor se entiende mejor si lo ubicamos como una **bisagra** entre el mundo de la **construcción** de *software* (*build/package*) y el de la **ejecución** (*deploy/run*). Por un lado, recibe artefactos producidos por CI que incorporan **metadatos, firmas y SBOM**; por el otro, los sirve a los consumidores bajo **condiciones** que resguardan la integridad del sistema: autenticación y **RBAC** para definir “quién puede qué”, **políticas** para determinar “qué se permite consumir y cuándo” y **auditoría** para reconstruir “qué pasó, quién lo hizo y con qué evidencia”.

Un concepto cardinal es la distinción entre **tag** y **digest**. El **tag** es una etiqueta **mutable** orientada a facilitar el trabajo humano (por ejemplo, **v1.5.0, release, 2025-09-01**). El **digest** es un **hash inmutable** del contenido (por ejemplo, **sha256: ...**) que identifica de forma única al artefacto. Para **trazabilidad, reproducibilidad y auditoría**, la identidad que importa es la del **digest**. Esta idea modifica la noción de “versión”: en lugar de asociar la versión a un nombre que puede moverse, se asocia al **contenido mismo**. De allí se desprende un principio clave que se trabajará a lo largo del curso: **desplegar por digest** y utilizar los **tags** como marcadores de **estado** y de **comunicación humana** (por ejemplo, **tested, approved, release**).

Harbor aporta además capacidades de **seguridad de la cadena de suministro**. Integra **escaneo de vulnerabilidades** (con un escáner como *Trivy*), soporte para **firma y verificación** (por ejemplo, con *cosign* del ecosistema Sigstore) y administración de **políticas** que pueden **impedir el consumo** de artefactos cuando no cumplen ciertos umbrales (por ejemplo, presencia de vulnerabilidades críticas no mitigadas). Estas funciones permiten pasar de una gestión basada en la confianza informal a una gestión sustentada en **evidencias verificables**.

Otra dimensión conceptual es la **multi-tenencia**: Harbor organiza el trabajo en **proyectos** (unidades lógicas que agrupan repositorios, políticas, cuotas y roles). Este esquema admite separar equipos, aplicaciones o entornos (desarrollo, pruebas, producción) y aplicar **gobernanza** diferenciada según riesgos y

responsabilidades. Complementariamente, el registro puede operar como **proxy cache** de *registries* públicos (acercando dependencias y mejorando resiliencia) y como plataforma de **replicación** hacia otros *registries* (para estrategias multi-región o multi-nube).

La **observabilidad** y la **auditoría** terminan de sostener su rol. Observabilidad implica comprender tendencias de uso (extracciones, crecimiento por proyecto, tiempos de escaneo), mientras que la auditoría conserva información sobre acciones relevantes (publicaciones, eliminaciones, cambios de etiquetas). Ambas dimensiones habilitan conversaciones informadas con los equipos de seguridad y cumplimiento, que valoran las evidencias objetivas por sobre relatos o supuestos.

De cara al aprendizaje, lo importante no es memorizar términos aislados, sino entrenar la **lectura de evidencias** y su interpretación en contexto: ¿qué significa que una imagen esté “aprobada” si el **digest** no coincide con el que se probó?, ¿qué riesgos implica promover por **tag**?, ¿cuáles son los **umbrales** razonables para permitir o bloquear el consumo de artefactos según criticidad de vulnerabilidades? Estas preguntas orientan el resto del manual y preparan al estudiantado para tomar decisiones fundamentadas.

Recursos para profundizar:

- Harbor Authors © (2025). Sitio oficial de Harbor. <https://goharbor.io/>
- Harbor Authors © (2025). Documentación principal (Harbor 2.13): <https://goharbor.io/docs/main/>
- The Linux Foundation ® (2025). CNCF — Proyecto Harbor: <https://www.cncf.io/projects/harbor/>
- The Linux Foundation ® (2025). Open Container Initiative — Portal: <https://opencontainers.org/>

2. Arquitectura lógica en Kubernetes y conceptos de exposición

Sin entrar en recetas operativas, es útil contar con un mapa **lógico** de los **componentes** que conforman Harbor en un entorno Kubernetes. En el plano conceptual, el servicio se compone de: **Core/API** (gestiona la lógica de negocio y la autenticación), **Portal** (interfaz web), **Registry** (implementa la distribución OCI), **Jobservice** (coordina tareas internas como escaneo, replicación y limpieza), **Base de datos** (almacena metadatos) y **Redis** (cache/colas/sesiones). A este conjunto se

suma un **escáner** —habitualmente *Trivy*— que provee los análisis de vulnerabilidades.

En Kubernetes, la **exposición** se realiza mediante un **controlador de Ingress** que encamina el tráfico hacia los servicios internos y termina **TLS**. Es correcto pensar en el Ingress como una pieza que permite **nombrar** y **proteger** el servicio hacia el exterior, y no como un detalle de implementación. En la misma línea, el manejo de **certificados** mediante un orquestador como **cert-manager** debe considerarse parte del diseño: al automatizar la emisión y renovación, se reduce el riesgo de interrupciones y se fortalecen las garantías de seguridad de extremo a extremo.

El **almacenamiento** es otro concepto crucial. El **Registry** y el escáner conservan información persistente (capas, metadatos, bases de firmas/CVEs); en términos teóricos, esto obliga a pensar en **clases de almacenamiento** (“StorageClass”) con prestaciones adecuadas para volumen, rendimiento y durabilidad. La capacidad no se dimensiona “a ojo”: se relaciona con políticas de **retención** y con hábitos de construcción (cantidad de versiones simultáneas, tamaño medio de capas, frecuencia de promoción).

La **variable de contexto** externa —que en muchas implementaciones aparece como `externalURL`— encapsula un principio: el sistema debe **saberse a sí mismo** y comunicar a los clientes su **identidad pública**, que incluye el **FQDN** y el **esquema** (HTTPS). Este dato no es cosmético: afecta la generación de tokens y la usabilidad de las integraciones, por lo que, a nivel conceptual, se recomienda definirlo con claridad desde el inicio para evitar confusiones posteriores.

La **replicación** y el **proxy cache** amplían el panorama arquitectónico. La primera consiste en distribuir artefactos hacia otros *registries* —quizás ubicados en otras regiones, nubes o dominios de seguridad— con **filtros** por proyectos o patrones. En la teoría, importa analizar **qué** se replica y **por qué** (proximidad a consumidores, resiliencia, independencia de dominios), además de **cómo** se mantienen las garantías de integridad (por ejemplo, al verificar firmas en el destino). El **proxy cache**, por su parte, se entiende como un mecanismo de **acercamiento** de dependencias públicas, que mejora tiempos de compilación y reduce la dependencia de terceros; en el plano conceptual, se evalúa por su impacto en **performance**, **resiliencia** y **gobernanza** (qué fuentes se permiten cachear).

Finalmente, el **modelo de proyectos y repositorios** merece una mirada arquitectónica. Los **proyectos** son contenedores lógicos de políticas, cuotas y roles; los **repositorios** agrupan artefactos por **nombre lógico** y se etiquetan con **tags**. Esta estructura permite implementar **separación de obligaciones** (por equipos o por entornos), reforzar el **principio de menor privilegio** y reducir el **acoplamiento** entre productoras y consumidoras de artefactos. La **auditoría** y las **métricas** complementan el diseño, habilitando una lectura sistémica del uso y la evolución del registro.

Recursos para profundizar:

- Harbor Authors © (2025). Documentación de Harbor (sección principal).
<https://goharbor.io/docs/main/>
- GitHub © (2025). Helm chart de Harbor (referencia conceptual):
<https://github.com/goharbor/harbor-helm>
- The cert-manager Authors © (2025). Documentación de cert-manager:
<https://cert-manager.io/docs/>
- Ingress NGINX (documentación del proyecto):
<https://kubernetes.github.io/ingress-nginx/>

3. Gobierno y seguridad de la cadena de suministro

La seguridad en registros de artefactos debe concebirse como una **defensa en capas** que combina **gobernanza** (quién define las reglas), **controles preventivos** (qué condiciones se exigen antes de permitir consumo) y **controles detectivos** (qué información se observa para identificar riesgo). En Harbor, el **gobierno** se materializa a través del **modelo de proyectos** y del **RBAC**: cada proyecto define visibilidad (público/privado), **roles** (administración, desarrollo, lectura) y **cuotas** (límite de consumo). Esta segmentación evita tanto el acceso excesivo como la creación de silos impenetrables, y habilita el principio de **menor privilegio**.

En el plano **preventivo**, dos bloques son centrales. Primero, la **firma** de artefactos, que otorga garantías de **integridad** y **autoría**. En términos teóricos, esto significa que cada artefacto lleva consigo una evidencia criptográfica que permite verificar que **no fue alterado** y que proviene de una **identidad reconocida** (ya sea por claves tradicionales o por modelos **keyless** basados en identidad federada). Segundo, el **escaneo** de vulnerabilidades, que identifica **exposición conocida** de los componentes que forman un artefacto. Conceptualmente, el escaneo debe regirse por **políticas** con **umbrales** claros (por ejemplo, bloquear consumo cuando se detectan vulnerabilidades **críticas** sin mitigación).

En el plano **detectivo**, adquiere relevancia el **SBOM** (Software Bill of Materials), que proporciona una **lista de materiales** del artefacto: componentes y versiones. Esta evidencia acelera la **respuesta a incidentes** al permitir consultas informadas (qué artefactos incluyen una librería afectada, en qué estado del ciclo se encuentran, dónde se consumen). La teoría sugiere estandarizar un formato (como **SPDX** o **CycloneDX**) y adjuntar el SBOM como artefacto asociado para que viaje con el contenido.

Los **controles de admisión** en Kubernetes (por ejemplo, con **Gatekeeper/OPA** o **Kyverno**) amplían el perímetro de seguridad: aun si en el registro existen políticas, es sano que los clústeres **verifiquen** que solo se ejecuten **imágenes firmadas y aprobadas** provenientes de los **proyectos** correctos. Desde la teoría, esto refuerza la *segregación de responsabilidades*: el registro **publica y certifica**; la plataforma **valida y aplica**.

Como marco general, **SLSA** (Supply-chain Levels for Software Artifacts) ayuda a **ordenar** prácticas y aspiraciones: desde niveles básicos (artefactos **inmutables**, registros de construcción **verificables**, **procedencia** adjunta) hasta niveles avanzados (*builders* **aislados**, **transparencia** pública, múltiples validadores). En todos los casos, Harbor funciona como **repositorio de evidencias**, donde se concentran artefactos y metadatos que facilitan auditorías internas y externas.

El enfoque teórico también invita a **gestionar excepciones**: no todas las vulnerabilidades ameritan bloquear releases; algunas requieren **aceptación** explícita, **plazos** de remediación y **documentación** del riesgo residual. El valor de Harbor reside en que esas decisiones se toman sobre **evidencias** objetivas (reportes de escaneo, firmas, SBOM, auditoría) y no sobre percepciones.

Recursos para profundizar:

- Trivy — documentación: <https://trivy.dev/>
- Sigstore — Cosign (documentación): <https://docs.sigstore.dev/cosign/>
- SPDX — sitio oficial: <https://spdx.dev/>
- CycloneDX — sitio oficial: <https://cyclonedx.org/>
- Kyverno — documentación: <https://kyverno.io/docs/>
- Gatekeeper/OPA — documentación: <https://open-policy-agent.github.io/gatekeeper/website/docs/>

4. Versionado, reproducibilidad y promoción inmutable

Pensar el **versionado** desde la teoría implica abandonar la idea de que el **tag** “es la versión”. El **tag** es un **nombre mutable** que se mueve; la verdadera versión, en términos materiales, es el **digest**. Este desplazamiento conceptual, aunque simple, tiene impactos profundos en la forma de **probar, aprobar, promover y revertir** artefactos.

La **reproducibilidad** se sostiene cuando el proceso de despliegue **apunta al digest**, ya que ello garantiza que el contenido ejecutado coincide con el

contenido probado. Los **tags** se convierten en **marcadores de estado** y de comunicación (por ejemplo, **snapshot**, **tested**, **release**, **approved**) y pueden existir en **paralelo** sin alterar la referencia material. Desde esta perspectiva, la **promoción** entre entornos no requiere **reconstruir**; basta con **eleva el estado** del **mismo digest**, manteniendo evidencia coherente de su camino (reportes de escaneo, atestaciones de firma, SBOM).

Una teoría del versionado responsable también contempla la **nomenclatura**: evitar etiquetas ambiguas como **latest** en contextos críticos; adoptar convenciones que comuniquen el **estado** sin inducir a error; y documentar **cuándo** un tag puede cambiar de referencia y **quién** autoriza ese cambio. La **política de rollback** debe definirse en términos de **estado/digest**: ¿qué implica revertir?, ¿qué evidencias se revalidan?, ¿cómo se registran las decisiones para auditoría?

Asimismo, la reproducibilidad depende de decisiones en etapas **anteriores**: elección de **imágenes base** con identidad estable (idealmente por *digest*), gestión de **dependencias** mediante archivos de bloqueo, captura de **metadatos** de construcción (origen, commit, fecha, herramientas). El valor de Harbor reside en **conservar** y **exponer** parte de esa evidencia, de modo que las decisiones no dependan de supuestos.

Finalmente, el enfoque teórico propone **criterios de promoción** que combinen **calidad** y **seguridad**: además de pruebas funcionales y de integración, exigir **evidencias** de escaneo “en verde”, **firma válida** y **SBOM** adjunto. Esta “puerta de promoción” formaliza el pasaje entre estados y reduce arbitrariedades.

Recursos para profundizar:

- OCI — Image Specification: <https://specs.opencontainers.org/image-spec/>
- OCI — Distribution Specification: <https://specs.opencontainers.org/distribution-spec/>
- CNCF — Harbor Project Journey Report: <https://www.cncf.io/reports/harbor-project-journey-report/>
- ORAS (OCI Registry as Storage): <https://oras.land/docs/>

5. Integración conceptual con CI/CD

En la articulación con **CI/CD**, es útil imaginar un **contrato** entre las etapas que **producen** artefactos y las que los **consumen**. **CI** es responsable de generar el artefacto junto con sus **evidencias** adjuntas (firma, SBOM, resultados de escaneo),

y de **publicarlo** en el proyecto correspondiente de Harbor. **CD** es responsable de **verificar** esas evidencias, de **promover por digest** y de **registrar** las decisiones asociadas al cambio de estado.

Este contrato se apoya en el modelo de **roles** y **credenciales**. A nivel teórico, conviene evitar el uso de cuentas personales y favorecer identidades técnicas con **alcance limitado** (por ejemplo, cuentas de robot asociadas a proyectos específicos) para minimizar superficies de riesgo y cumplir el **principio de menor privilegio**. En paralelo, se deben enunciar **puntos de control**: sin firma válida no se promueve; sin escaneo dentro de umbrales establecidos no se habilita el consumo; sin SBOM adjunto no se puede responder a incidentes con la celeridad requerida.

La **comunicación** entre productores y consumidores se organiza mediante **eventos** (por ejemplo, notificaciones del registro cuando un artefacto cambia de estado) y mediante **acuerdos de nomenclatura** (tags de estado estandarizados). En un entorno distribuido, la **replicación** de proyectos hacia otros *registries* cumple la función de **acercar** artefactos a los puntos de consumo, sosteniendo **consistencia** y **verificación** de evidencias. El **proxy cache** de fuentes externas, a su vez, reduce **latencia** y **dependencias** durante la construcción.

La perspectiva teórica refuerza la importancia de la **observabilidad** del *pipeline*: métricas de **tiempo de ciclo** desde que un *digest* es producido hasta que es promovido; tasas de **rechazo** por políticas; y calidad de la **evidencia** adjunta. Estas métricas no son decorativas: retroalimentan la mejora continua y orientan inversiones (por ejemplo, en almacenamiento, en infraestructura de escaneo o en red).

Por último, el **diseño de responsabilidades** es parte del contrato: quién define las políticas de promoción, quién gestiona las **excepciones** (y cómo se documentan), quién administra **cuotas** y **retención**, y cómo se realiza el **resguardo** de metadatos relevantes. El objetivo para el estudiantado es comprender que la integración CI/CD con Harbor no es una secuencia de comandos, sino un **sistema de acuerdos** sustentado en evidencias y políticas explícitas.

Recursos para profundizar:

- Harbor — documentación de administración:
<https://goharbor.io/docs/main/administration/>
- GitHub Actions — Publicación de imágenes (referencia conceptual):
<https://docs.github.com/actions/publishing-packages/publishing-docker-images>
- GitLab CI — Imágenes y registros (referencia conceptual):
https://docs.gitlab.com/ee/ci/docker/using_docker_build.html
- Harbor — webhooks y eventos (referencia conceptual):
<https://goharbor.io/docs/main/administration/configure-webhooks/>

6. Operación, gobernanza y riesgo

Aunque este manual es teórico, resulta imprescindible incorporar una mirada de **operación** y **gobernanza**. Operar un registro implica tomar decisiones sobre **vida útil** de los artefactos (políticas de **retención**), higiene del repositorio (**garbage collection** de capas huérfanas) y administración de **cuotas** para evitar crecimiento desordenado. Estas políticas no son solo operativas: forman parte de la **gobernanza**, porque condicionan costos, performance, y la posibilidad de reconstruir el pasado con evidencia suficiente.

La **auditoría** es otro pilar. Registrar y poder consultar **quién** publicó, **qué** publicó, **cuándo** y **con qué evidencia** permite explicar decisiones y responder ante cuestionamientos internos o regulatorios. A su vez, la **observabilidad** abre el tablero: métricas sobre **extracciones**, **errores** de validación, **tiempos** de escaneo, **crecimiento** por proyecto. Estas métricas alimentan conversaciones de capacidad (almacenamiento, cómputo para escaneo) y de **riesgo** (picos anómalos de descargas, accesos fuera de horario, consumo desde ubicaciones inesperadas).

La **gestión de TLS** y de **identidades** compone el plano de seguridad perimetral. Conceptualmente, un esquema con certificados válidos, gestión automatizada de expiración y prácticas de rotación de credenciales reduce exposición a fallas y mejoras en experiencia de uso. En paralelo, un **catálogo de políticas** (qué se permite consumir, bajo qué umbrales y con qué excepciones) formaliza el contrato social entre desarrollo, seguridad y operación.

En entornos multi-región o multi-nube, la **replicación** planificada y la **estrategia de recuperación ante desastres (DR)** se vuelven esenciales. Teóricamente, la replicación cumple objetivos de **proximidad**, **resiliencia** y **soberanía**, pero también introduce preguntas de **consistencia** y de **autoridad** (¿qué *registry* es la fuente de verdad para cada proyecto?). Un diseño sólido explicita **límites de responsabilidad** y verifica la **integridad** en el destino (por ejemplo, a través de verificación de firmas).

Por último, una visión de **riesgo** invita a ponderar no sólo **vulnerabilidades técnicas**, sino también riesgos **organizacionales** (por ejemplo, dependencia de personas clave para aprobar promociones), **contractuales** (acuerdos de servicio con equipos que consumen artefactos) y **económicos** (costos de almacenamiento y cómputo asociados al escaneo). El estudiantado debería salir con la capacidad de evaluar **intercambios** ("*trade-offs*") y de proponer **mitigaciones** razonables, fundamentadas en evidencia y en prioridades del negocio.

Recursos para profundizar:

- Harbor Authors © (2025). Harbor — administración y operación:
<https://goharbor.io/docs/main/administration/>
- The Linux Foundation ® (2025). CNCF — Proyecto Harbor:
<https://www.cncf.io/projects/harbor/>
- The cert-manager Authors © (2025). Cert-manager — documentación:
<https://cert-manager.io/docs/>
- Gatekeeper — documentación:
<https://open-policy-agent.github.io/gatekeeper/website/docs/>

Bibliografía

- Cloud Native Computing Foundation. (s. f.). **Harbor — CNCF Project**. Recuperado de <https://www.cncf.io/projects/harbor/>
- Harbor Project. (s. f.). **Sitio oficial**. Recuperado de <https://goharbor.io/>
- Harbor Project. (2025). ****Documentación 2.1**